



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE - UFRN
Centro de Tecnologia - CT

Programa De Pós-Graduação Em Engenharia
Elétrica e De Computação - PPgEEC/UFRN

***AcademicAssistant: a multi-agent system for
academic research, review and organization***

Angelo Leite Medeiros de Góes (20251012333)
William Marcelino Costa do Nascimento (20251026230)

Natal (RN), julho de 2026

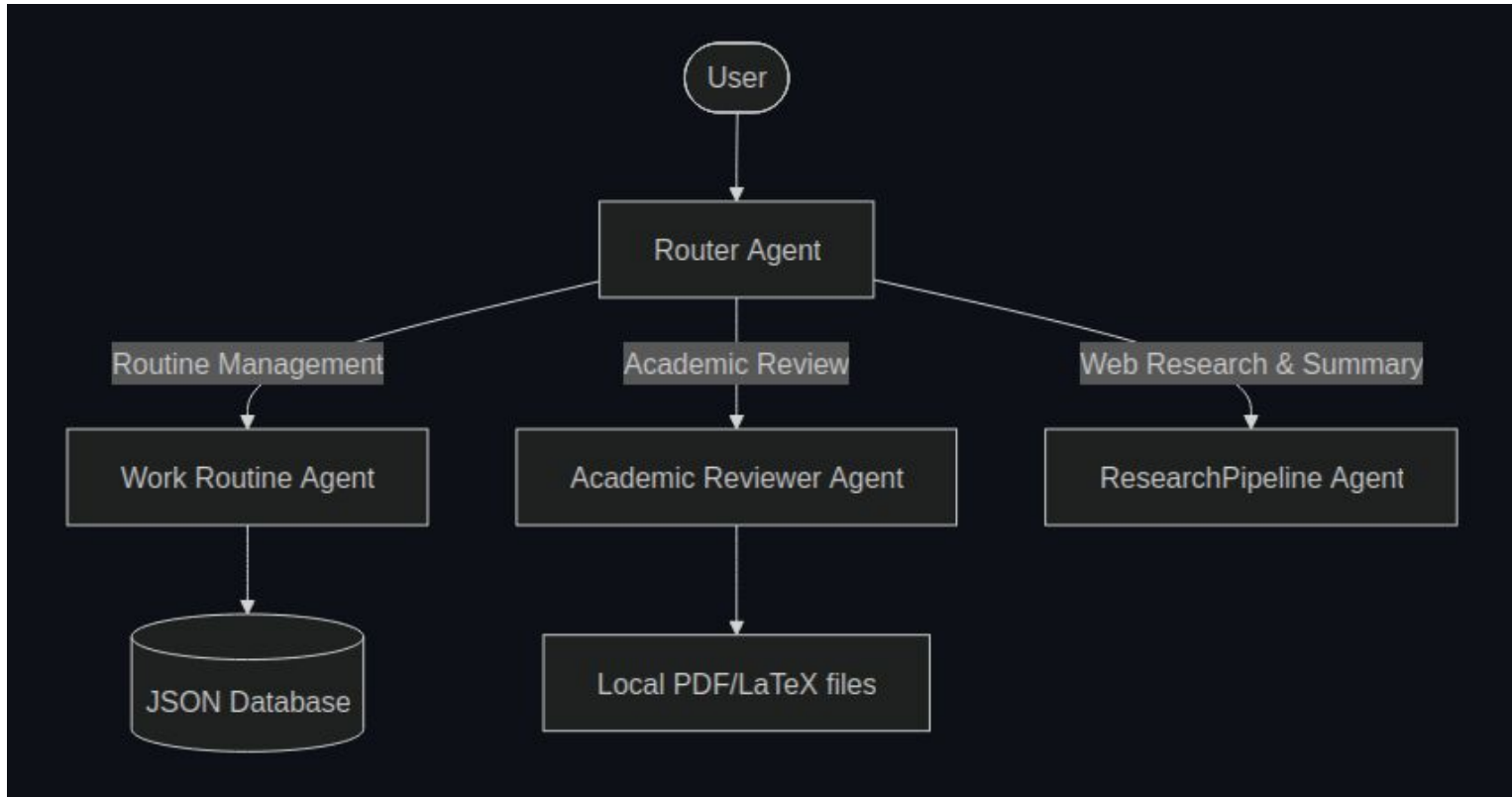
AcademicAssistant

Multi-agent assistant for a researcher's day-to-day work

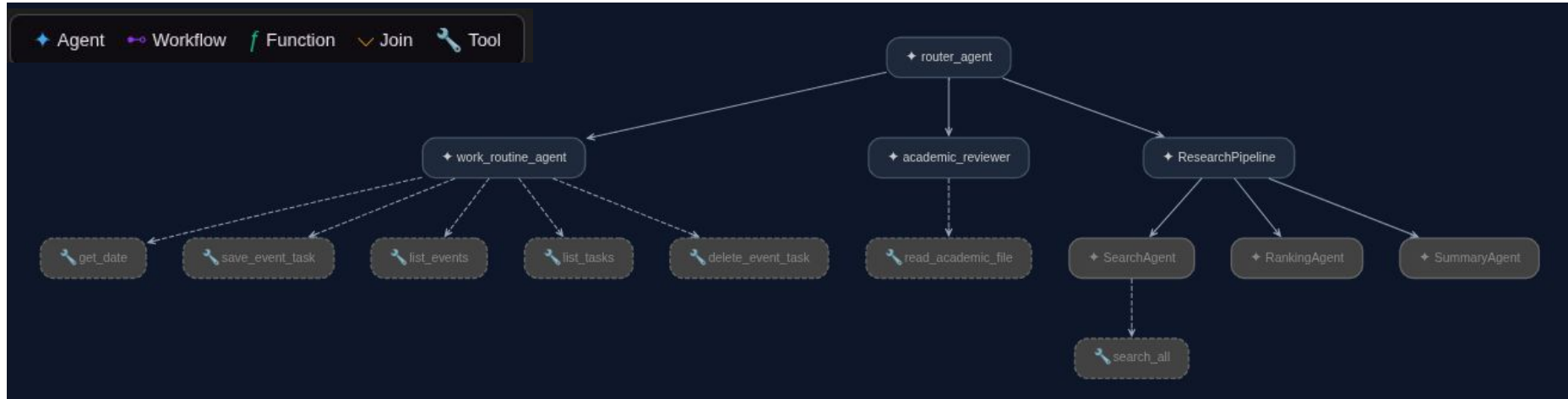
1. Jot down and list tasks
2. Grammar corrections for texts
3. Quick search and summary of scientific literature



System Architecture



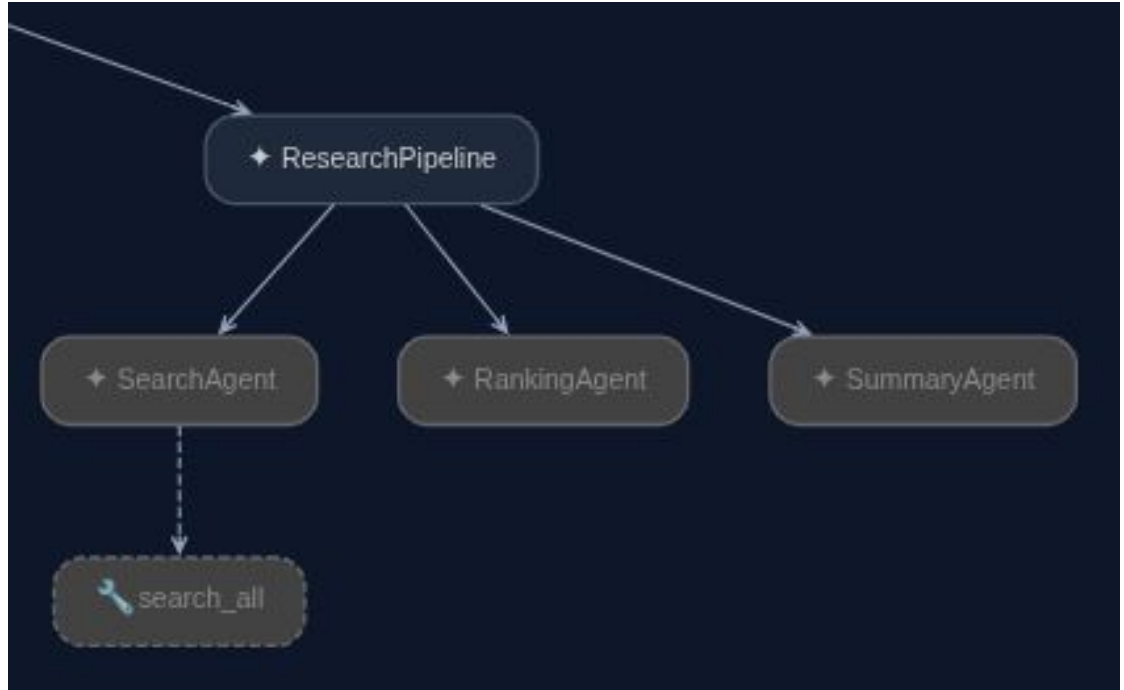
System Architecture



ResearchAgent (orchestrator)

Pipeline:

1. Search agent
 - a. “search_all” tool
2. Ranking agent
3. Summary agent



SearchAgent

Query N articles from:

- arXiv API
- OpenAlex API
- Crossref API

then output JSON to

{papers}



OpenAlex



```
papers.append(  
    {  
        "title": paper.title,  
        "abstract": paper.summary,  
        "authors": [a.name for a in paper.authors],  
        "year": paper.published.year,  
        "url": paper.entry_id,  
        "source": "arxiv",  
    }  
)
```

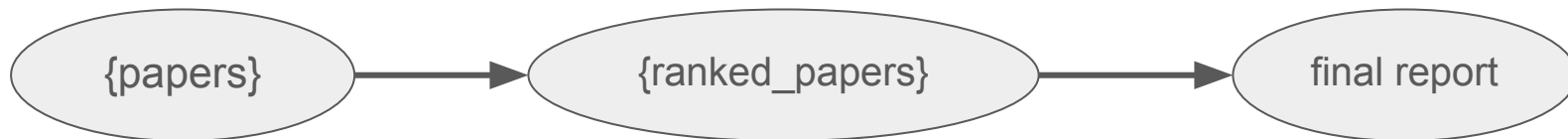
```
5 search_agent = LlmAgent(  
6     name="SearchAgent",  
7  
8     model="gemini-2.5-flash",  
9  
10    instruction="""  
11    Search scientific literature.  
12  
13    Always use search_all.  
14  
15    Return ONLY the list of papers.  
16    """,  
17  
18    tools=[search_all],  
19  
20    output_key="papers",  
21 )
```

RankingAgent and SummaryAgent

```
3 ranking_agent = LlmAgent(  
4     name="RankingAgent",  
5  
6     model="gemini-2.5-flash",  
7  
8     instruction="""  
9     The following papers were retrieved.  
10  
11     {papers}  
12  
13     Tasks  
14  
15     - remove duplicates  
16     - rank by relevance  
17     - preserve every field  
18     - do not summarize  
19  
20     Return ONLY the ranked list.  
21     """,  
22  
23     output_key="ranked_papers",  
24 )
```



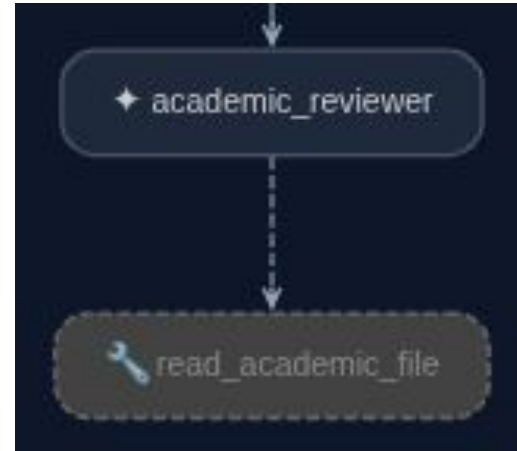
```
3 summary_agent = LlmAgent(  
4     name="SummaryAgent",  
5  
6     model="gemini-2.5-flash",  
7  
8     instruction="""  
9     Using these ranked papers:  
10  
11     {ranked_papers}  
12  
13     Generate  
14  
15     - overview  
16     - research themes  
17     - top 5 papers  
18     - recommended reading order  
19     - future research directions  
20  
21     Be concise.  
22     """,  
23  
24     output_key="final_report",  
25 )
```



Academic reviewer Agent

Performs a grammatical review of an academic text

- Tool:
 - a. Read document text
 - i. File .pdf
 - ii. File .tex



Academic reviewer Agent

```
from google.adk.agents.llm_agent import Agent
from .tools import read_academic_file
```

```
academic_reviewer = Agent(
    model='gemini-2.5-flash',
    name='academic_reviewer',
    description='Especialista em revisão de texto',
    instruction=(
        "You are a specialist in academic text review.",
        "Your role is to analyze the text submitted by the user",
        "and find grammatical errors, spelling mistakes, and clarity issues.",
        "For each error or suggestion identified, provide the following information in Portuguese:",
        "***Trecho original:** [The entire original text snippet]",
        "***Explicação:** [Clear explanation of why it's an error or suggestion]",
        "***Sugestão:** [Suggested correction or improvement]",
        "Please use a horizontal rule `---` to separate different sections.",
        "Rules:\n"
        "1. If the user provides a local file path, read the file content.",
        "2. Do not modify the original text directly. Only provide corrections in the response.",
        "3. Language Rule: By default, converse and provide explanations in Portuguese."
    ),
    tools=[read_academic_file]
)
```

```
def read_academic_file(file_path: str) -> dict:
    """Lê o conteúdo de um arquivo acadêmico local (.pdf ou .tex) e retorna o seu texto co

    Esta ferramenta deve ser usada sempre que o usuário solicitar a revisão de um arquivo
    especificando um caminho (.pdf ou .tex).

    Args:
        file_path: O caminho do arquivo (.pdf ou .tex) no sistema de arquivos local.

    Returns:
        dict: Um dicionário contendo o status da leitura ("success" ou "error") e o texto
        """
    path = os.path.abspath(file_path.strip())
```

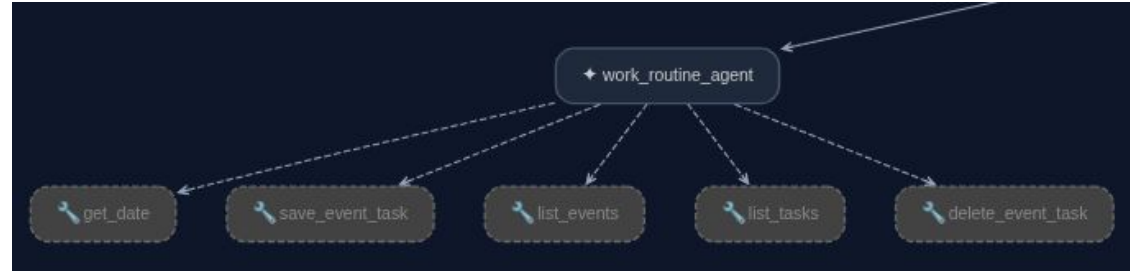
```
ext = os.path.splitext(path)[1].lower()
if ext == ".tex":
    try:
        # ...
    except:
        pass
elif ext == ".pdf":
    try:
        # ...
    except:
        pass
else:
    return {
        "status": "error",
        "message": f"Formato de archivo no soportado: {ext}"
    }
```

```
elif ext == ".pdf":
    try:
        from pypdf import PdfReader
        reader = PdfReader(path)
        content = ""
        for page in reader.pages:
            text = page.extract_text()
            if text:
                content += text + "\n"
        return {
            "status": "success",
            "format": "pdf",
            "content": content
        }
    except:
        return {
            "status": "error",
            "format": "pdf",
            "content": "Error: Invalid PDF file."
        }
```

Work routine Agent

Manages user tasks and events using a JSON file to store information.

- Tools:
 - a. Get now date
 - b. Save event or task
 - c. Get event or task
 - d. Delete event or task



Work routine Agent

```
# Path to the shared JSON database file located in the parent directory
DB_FILE = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "database.json"))

def load_data() -> List[Dict[str, Any]]:
    """Internal helper to load all registered tasks and events from the JSON database."""
    if not os.path.exists(DB_FILE):
        return []
    try:
        with open(DB_FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    except (json.JSONDecodeError, IOError):
        return []

> def save_data(data: List[Dict[str, Any]]) -> None:--

> def get_date() -> dict:--

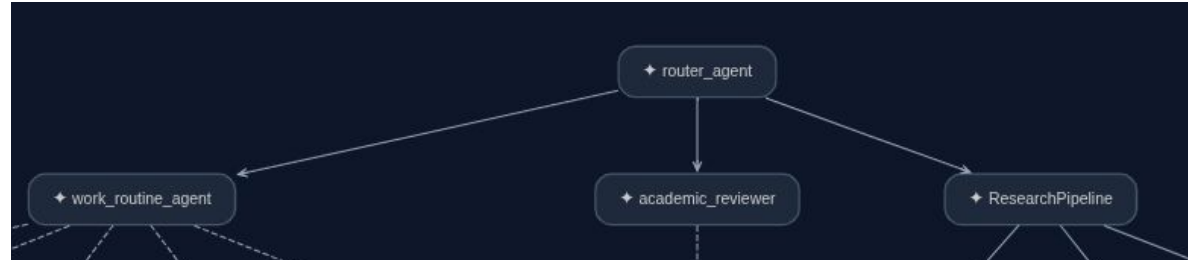
> def save_event_task(titulo: str, tipo: str, data: str, descricao: str) -> dict

> def list_events(date: str) -> dict:--
```

```
[
  {
    "titulo": "Rodar os experimentos da minha pesquisa",
    "tipo": "tarefa",
    "data": "2026-07-02",
    "descrição": "Rodar os experimentos do tcp e do udp"
  },
  {
    "titulo": "Falar com Angelo",
    "tipo": "tarefa",
    "data": "2026-07-02",
    "descrição": "Morning"
  }
]
```

Router Agent

It receives the prompt and decides which agent should respond to the request.



Router Agent

```
# Coordinator/Routing Agent
router_agent = Agent(
    model='gemini-2.5-flash',
    name='router_agent',
    description='Coordenador principal que direciona as solicitações do usuário para os especialistas',
    instruction=(
        "You are the main coordinator. Your job is to understand the user's intent and route the conversation to the appropriate specialist agent. Follow these rules:\n"
        "- Delegate to `work_routine_agent` for anything related to work routine, calendar events, or scheduling.\n"
        "- Delegate to `academic_reviewer` for academic text review, grammar checks, or reading PDFs.\n"
        "- Delegate to `ResearchPipeline` for academic web research, literature search, ranking, or data analysis.\n"
        "Rules:\n"
        "1. Route immediately as soon as you identify the user's intent. Do not answer questions yourself."
    ),
    sub_agents=[work_routine_agent, academic_reviewer, research_pipeline]
)

# Expose router_agent as the root_agent for the app
root_agent = router_agent

from google.adk.apps import App

app = App(root_agent=root_agent, name="app")
```

Custos



Baixar o CSV

Serviço	Custo de uso ?	Programas de economia ?	Outras economias ?	↓ Subtotal	% de alteração ?
● Gemini API	R\$ 1,38	—	R\$ 0,00	R\$ 1,38	↑ 60%

Obrigado!

